P3570R1
optional variants in sender/receiver

Draft Proposal, 2025-02-10

Author:
Fabio Fracassi (Code University of Applied Sciences) fabio@fracassi.de

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Some async interfaces for concurrent queues would be well served by using `std::optional` when used in a coroutine context. However when using sender/receiver interfaces directly the variant style visitor based on overloading has several benefits. This paper explores if and how we can get the best interface for each use case.

Table of Contents

§ 1. Motivation

This paper is a reaction to the Library Evolution(LEWG) review of the Concurrent Queues paper [P0260R13], specifiaclly the review of revision 13. The return type of `async_pop()` has been contentious for a while, and is has been blocking consensus of a otherwise highly anticipated facility.

The contention stems from the fact that the `async_pop()` (and other senders) can be used in two different contexts. It can be used in chain of sender adapters or can be awaited on in a coroutine. During the review it became clear that depending on the context LEWG would prefer a slightly different usage pattern. Consider the following examples (adapted from C++ Concurrent Queues § Examples) for the prefered usage:

```
1 auto coro_grep(stdx::coqueue<std::fs::path>& file_queue, std::string neec
2   -> stdp::task<void>
3 {
4   while (auto fname = co_await file_queue.async_pop()) {
5     grep_file(*fname, needle);              // queue has data
6   }
7                                 // queue closed
8   co_return;
9 }
```

particularly note line 4 and 5 in the coroutine example above and line 6 to 10 in the S/R example below

```
1  stdexec::sender auto sr_grep(auto scheduler
2                , stdx::coqueue<std::fs::path>& file_queue, std::string ne
3  {
4    return stdexec::repeat_effect_until(
5      stdexec::starts_on(scheduler
6        , files.async_pop()
7        | stdp::overload(
8          []() -> bool { return true; },                  // queue closed
9          [needle](std::fs::path const& fname) -> bool {  // queue has dat
10           return grep_file(fname, needle);
11         });
12     ));
13 }
```

We can make either of these interfaces work, depending on the completion signatures that the sender that is returned from `async_pop()` supports. To support the ideal coroutine interface the `async-pop-sender` should have the completion signature `set_value_t(std::optional<T>)`, to support the ideal S/R interface the completion signatures would need to be `set_value_t()` and `set_value_t(T)`. Whichever we support slightly penalizes the interface for the other, see Appendix A (Workarounds).

With this paper we want to explore if and how we could support the ideal interface for both contexts.

§ 2. Exploration

To understand a path toward a solution we need to take a short look into how senders/receivers interact with coroutines.

The sender is not directly used in the coroutine, but is transformed into an `awaitable`. The transformation is initiated by the coroutine task type (as proposed in [P3552R0]) which uses S/Rs `std::as_awaitable` to do the actual transformation mechanics.

§ 2.1. Option 1: custom awaiter

The simplest option is to exploit that `std::as_awaitable` provides a customization point, allowing a sender to provide its own custom `awaiter` type.

This is strightforward, and allows us to get the ideal interface for both contexts. It involves very little machinery, and in fact even bypasses most of the S/R machinery, most likely allowing for good compile times and runtime efficiency.

However the custom awaiter will only be used if the `async-pop-sender` is used directly. The result of `async_pop` can not be adapted by any sender adapters. This will severely limit implementation freedom, and would probably cause some complicated code duplication, e.g. in the coroutine task type.

it would also make the abstraction brittle for the user, since the preferred interface would only work on unadapted senders.

§ 2.2. Option 2: generalized value transform

The second option is to hook into the `std::as_awaitable` machinery in a more generic fashion.

Currently the C++ Working draft spells out several potential transformations in [exec.as.awaitable], that are tried one after the other. The one that allowes senders with a single value completion signature to be used seamlessly is described in §7.3 which consist of two exposition only facilities, an `awaitable-sender` concept and a `sender-awaitable` type.

We could add another protential transformation right after that single sender step to `std::as_awaitable`. This step would take an exposition only type trait `completion-adapter` that provides a sender adapter (for the motivating usecase from [P0260R13] that adapter would be `into_optional` as described in [P3552R0] Sec 4.4). `as_awaitable` would now check if the original sender that has been adapted with the transformation provided by the `completion-adapter` trait conforms to the `awaitable-sender` concept in which case it would provide the `sender-awaitable` for the adapted sender.

The `into_optional` transform takes a sender with a `set_value_t()` and `set_value_t(T)` and changes that into a `set_value_t(std::optional<T>)` completion signature.

There are three options for how we could use the `completion-adapter` trait:

2.2.1. Option 2.a1: Always transform

The type trait is exposition only and defaults to `into_optional`. No specialisations are provided. `as_awaitable` will alway try the `into_optional` transformation.

This means all senders that have compatible value completion signatures will automatically be usable in coroutines with the `std::optional<T>` return type. Users wont be able to customize the behavior. We would have to fully specify `into_optional`

2.2.2. Option 2.a2 Opt-Out

Same as § 2.2.1 Option 2.a1: Always transform, but with a opt-out mechanism.

The type trait is exposition only and defaults to `into_optional`. A specialisations is provided, that will query the sender type with a new environment property `get_await_completion_adapter`. The (original) sender can provide such a property like this:

```
1 struct env { auto query(get_await_completion_adapter_t) const -> std::fa
2 auto get_env() const noexcept -> env { return {}; }
```

In this case the trait specialisation will return a special type that `std::awaitable` uses to skip the adaption step.

This means all senders that have compatible value completion signatures will automatically be usable in coroutines with the `std::optional<T>` return type, but users could opt out in case the optional semantic do not make sense for a certain sender. We would have to fully specify `into_optional`

2.2.3. Option 2.b Opt-In

Same as § 2.2.1 Option 2.a1: Always transform, but only if the sender explicitly opts-in.

The type trait is exposition only and defaults to an exposition only type that will make `std::as_awaitable` skip the adaption step (i.e. same the behavior as the current status quo). A specialisations is provided, that will query the sender type with a new environment property `get_await_completion_adapter`, that will return the adapter type from the query.

The (original) sender can provide such a property like this:

```
struct env { auto query(get_await_completion_adapter_t) const -> into_c
auto get_env() const noexcept -> env { return {}; }
```

Note that in this approach the senders environment property provides a the adaper type directly. This gives users full control over which adaption they want, so the mechanism would not be restricted to a specific transformation.

This means that senders that have compatible value completion signatures will not automatically be usable in coroutines with the `std::optional<T>` return type, but sender providers could opt in when the optional semantic does make sense for a certain sender, as for example the motivating `async-pop-sender`.

§ 3. Proposal

A custom awaiter as described in § 2.1 Option 1: custom awaiter solves our imediate problem, but does so only for the specific case, and seems overly brittle.

§ 2.2.1 Option 2.a1: Always transform solves our problem in a generic fashion, but might cause new problems, because it automatically changes signatures of senders, that might be ill suited for this transformation. The same is true for § 2.2.2 Option 2.a2 Opt-Out, but it would give such ill suited senders a way out.

We propse to go with § 2.2.3 Option 2.b Opt-In. There is no clear evidence how often the `std::optional<T>` semantics are inappropriate for senders that otherwise fullfill the completion signatures, so we propose to go with the conservative option that will allow senders to opt-into the semantics when appropriate, but never automatically.

This Option also has the benefit of being fully generic, with very localized changes, and no need define the actual adaption type as part of either this paper or the lazy task type. Concurrent queues could either use a generic `into_optional` type or a custom adapter for that purpose. This also leaves the design space open for the concurrent queues to provide different adapters such as e.g. disengage_on_close or disengage_on_empty.

4. Wording

4.1. Header synopsis [version.syn]

```
#define __cpp_lib_exec_await_adapters <editor supplied value> // also in
<exec>
```

4.2. Execution control library [exec]

4.2.1. execution::get_await_completion_adapter [exec.get.await.adapt]

1. `get_await_completion_adapter` asks a queryable object for its associated awaitable completion adapter.
2. The name get_await_completion_adapter denotes a query object. For a subexpression `env`, `get_await_completion_adapter(env)` is expression-equivalent to `MANDATE-NOTHROW(as_const(env).query(get_await_completion_adapter))`.
3. `forwarding_query(execution::get_domain)` is a core constant expression and has value true.

4.2.2. Awaitable completion adapter trait [exec.trait.await.adapt]

[Drafting Q: does this deserve its own heading? If so were does this go?]

1. `template<class Sndr> concept has-queryable-await-completion-adapter // exposition only = sender<Sender> && requires(Sender&& sender) { { get_env(sender) }; { get_await_completion_adapter(stdexec::get_env(sender)) }; };`
2. Let `template<class Sndr> struct await-completion-adapter-trait` be an exposition only `Cpp17TransformationTrait` ([meta.rqmts]).
3. `await-completion-adapter-trait`'s member typedef type denotes the associated awaitable completion adapter if `Sndr` models `has-queryable-await-completion-adapter` otherwise `struct no-await-completion-adapter {};`

4.2.3. execution::as_awaitable [exec.as.awaitable]

(7.3) Otherwise, `sender-awaitable{expr, p}` if `awaitable-sender<Expr, Promise>` is true.

(7.4) Otherwise, [Drafting Q: Not sure how to word this as elegantly as the surrounding clauses] Let `AT` be `await-completion-adapter-trait<Expr>::type`.

(7.4.1) (void(p), expr) If `AT` is `no-await-completion-adapter`

(7.4.2) Otherwise Let `adapt-completion` be `AT{}`.

`sender-awaitable{adapt-completion(::std::forward<Expr>(expr)), p}` if `awaitable-sender<decltype(adapt-completion(::std::forward<Expr>(expr))), Promise>` is true.

(7.5) Otherwise, (void(p), expr).

5. Implementation Experience

§ 2.1 Option 1: custom awaiter, § 2.2.1 Option 2.a1: Always transform and § 2.2.3 Option 2.b Opt-In have been prototyped on top of [exec26]. The implementation is strightforward, see Appendix B (Prototype).

6. Acknowledgements

Dietmar Kühl, for his implementation of senders/receivers, and for talking through some of the initial ideas. Ian Petersen, for helping me with the opt-in mechanism.

Appendix A (Workarounds)

Workaround for the coroutine case if `async-pop-sender` supports `set_value_t()` and `set_value_t(T)`

```
1 while (auto fname = co_await file_queue.async_pop() | stdp::into_optiona
2   grep_file(*fname, needle);
3 }
```

The drawback here is that everyone needs to remember the additional `| stdp::into_optional()` for what is most likely a very common application programming usecase.

The code for sender/receiver if `async-pop-sender` supports `set_value_t(std::optional<T>)`

```
files.async_pop() | stdp::overload(
  [needle](std::optional<std::fs::path const>& fname) -> bool {
    if (!fname) { return false; }

    return grep_file(*fname, needle);
  });
```

This has two drawbacks, it introduces the potential for UB, since users could forget to check the optionals engaged state. The wrapping and unwrapping of the optional could also possibly introduce some overhead.

Concievably we could also introduce a special `overload` function for this case, which would get rid of the first drawback.

```
files.async_pop()
| stdp::overload_optional(
  []() -> bool { return true; },
  [needle](std::fs::path const& fname) -> bool {
    return grep_file(fname, needle);
  });
```

§ Appendix B (Prototype)

Prototype implementation based on [exec26]:

```
#include <beman/execution26/execution.hpp>
#include <beman/execution26/detail/queryable.hpp>
#include <optional>
#include <print>


namespace stdexec = beman::execution26;

```

```cpp
//-----------------------------------------------------------------------
// try out different options

//#define OPTION1(...)  __VA_ARGS__
#define OPTION1(...)

#define OPTION2b(...)  __VA_ARGS__
//#define OPTION2b(...)

//-----------------------------------------------------------------------
// Optional Sender Adapter (taken from P3552-task branch)
template <typename...> struct type_list {};

inline constexpr struct into_optional_t : beman::execution26::sender_
    template <stdexec::sender Upstream>
    struct sender {
        using upstream_t = std::remove_cvref_t<Upstream>;
        using sender_concept = stdexec::sender_t;
        upstream_t upstream;

        template <typename T> static auto find_type(type_list<type_li
        template <typename T> static auto find_type(type_list<type_li
        template <typename T> static auto find_type(type_list<type_li

        template <typename Env> auto get_type(Env&&) const {
            return decltype(find_type(stdexec::value_types_of_t<Upstr
        }
        template <typename... E, typename... S>
        constexpr auto make_signatures(auto&& env, type_list<E...>, t
            return stdexec::completion_signatures<
                stdexec::set_value_t(decltype(this->get_type(env))),
                stdexec::set_error_t(E)...,
                S...
                >();
        }
        template<typename Env>
        auto get_completion_signatures(Env&& env) const {
            return make_signatures(env,
                                   stdexec::error_types_of_t<Upstream
                                   std::conditional_t<
                                        stdexec::sends_stopped<Upstre
                                        type_list<stdexec::set_stoppe
                                        type_list<>>{}
```

```
                                    );
        }

        template <typename Receiver>
        auto connect(Receiver&& receiver) && {
            return stdexec::connect(
                stdexec::then(std::move(this->upstream),
                    []<typename...A>(A&&... a)->decltype(get_type(stc
                        if constexpr (sizeof...(A) == 0u)
                            return {};
                        else
                            return {std::forward<A>(a)...};
                }),
                std::forward<Receiver>(receiver)
            );
        }
    };

    template <typename Upstream>
    auto operator()(Upstream&& upstream) const -> sender<Upstream> {
} into_optional{};


namespace P3570_detail {
//-----------------------------------------------------------------------
// this type would become part of the standard
struct get_await_completion_adapter_t {
    template<typename T>
    auto operator()(T const& env) const noexcept
        requires requires(T&& t) { { t.query(std::declval<get_await_c
    {
        return env.query(*this);
    }
};
inline constexpr get_await_completion_adapter_t get_await_completion_
//-----------------------------------------------------------------------
// implementation detail, can needs to be cleaned up
struct no_tranform_t{ // implementation only tag type, no need to sta
    // member is not necessary, only there to
    template <typename Expr> auto operator()(Expr&& expr) const { ret
};
template<typename Sender> concept _has_env = requires(Sender&& sender
                            { stdexec::get_env(sender) };
```

```
95                          };
96
97  template<typename Sender, typename Env = Sender>
98  concept has_p3570_opt_in = ::beman::execution26::sender<Sender>
99                          && _has_env<Sender>
100                         && requires(Sender&& sender, Env&& env) {
101                             {
102                                 P3570_detail::get_await_completion_ac
103                             };
104                         };
105
106 // Exposition only trait type mentioned in the paper
107 template<typename Sender> struct _transform_trait { using type = no_t
108 template<typename Sender> requires has_p3570_opt_in<Sender>
109 struct _transform_trait<Sender> {
110     using type = decltype(P3570_detail::get_await_completion_adapter(
111 };
112 template<typename Sender> using _transform_trait_t = typename _transf
113
114 // I used an additional parameter to implement the transformation, bu
115 // the trait could just be used internally as well.
116 struct as_awaitable_t {
117     template <typename Expr, typename Promise, typename Transform = _
118     auto operator()(Expr&& expr, Promise& promise, Transform transfor
119         if constexpr (requires { ::std::forward<Expr>(expr).as_awaita
120             static_assert(
121                 ::beman::execution26::detail::is_awaitable<decltype(:
122                                                            Promise>,
123                 "as_awaitable must return an awaitable");
124             return ::std::forward<Expr>(expr).as_awaitable(promise);
125         } else if constexpr (::beman::execution26::detail::
126                                 is_awaitable<Expr, ::beman::executic
127                              not ::beman::execution26::detail::awaita
128             using TExpr = ::std::remove_cvref_t<decltype(transform(::
129             if constexpr ( std::is_same_v<std::remove_cvref_t<Transfc
130                         || not ::beman::execution26::detail::awaitabl
131                 return ::std::forward<Expr>(expr);
132             } else {
133                 return ::beman::execution26::detail::sender_awaitable
134             }
135         } else {
136             return ::beman::execution26::detail::sender_awaitable<Exp
137         }
```

```
138     }
139 };
140 inline constexpr ::P3570_detail::as_awaitable_t as_awaitable{};
141 }
142
143 //-----------------------------------------------------------------
144 // Fake concurrent queue to illustrate the mechanism
145
146 template<typename T_>
147 struct coqueue{
148     using T = int; // I just want to demonstrate the S/R and Coroutir
149                    // and make the fake queue as simple as prossible
150
151     struct pop_sender {
152         using sender_concept        = stdexec::sender_t;
153         using completion_signatures = stdexec::completion_signatures<
154                                                                       ,
155                                                                       >
156
157         OPTION1(
158         // Option 1: custom awaiter for pop_sender, works in the simp
159         //            (if somefuturestd::task does not adapt the senc
160         //           + No need to change any S/R machinery, works out
161         //           - any Sender adaption will break the special cas
162         //           - some complexity from implementing S/R operatic
163         //             duplicated in awaiter/await_resume
164
165         template<typename U, typename promise_t>
166         struct awaiter {
167             auto await_ready() -> bool { return {}; }
168             auto await_suspend(std::coroutine_handle<> p) -> std::cor
169             auto await_resume() -> std::optional<U> {
170                 return queue_.empty() ? std::optional<U>{}
171                                       : std::optional<U>{queue_.inter
172             }
173
174             promise_t promise_;
175             coqueue<U>& queue_;
176         };
177
178         template<typename promise_t>
179         auto as_awaitable(promise_t&& promise) {
180             std::println("Using option 1");
```

```
181             return awaiter<T, promise_t>{std::forward<promise_t>(prom
182         }
183         ) // /OPTION1
184
185         // this is the S/R mechanism for the "queue" (slideware versi
186         // Option 1 bypasses this part of the machinery in the corout
187         template<stdexec::receiver rcvr>
188         struct state {
189             using _result_t = T;
190             using _complete_t = void (state&);
191
192             rcvr rcvr_;
193             _complete_t* complete_;
194             coqueue<T>& queue_;
195
196             explicit state(rcvr&& r, coqueue<T>& queue)
197                 : rcvr_{std::move(r)}
198                 , complete_{[](state& self) {
199                     // here be synchronization
200                     if ( not self.queue_.empty()) { // this would che
201                         stdexec::set_value(std::move(self.rcvr_), sel
202                     } else {
203                         stdexec::set_value(std::move(self.rcvr_));
204                     }
205                 }}
206                 , queue_{queue}
207             {
208                 std::println("Using option 2 or S/R directly");
209             }
210
211
212             using operation_state_concept = stdexec::operation_state_
213             auto start() noexcept -> void {
214                 (*complete_)(*this);
215             }
216
217         // Option 2:
218         // this needs modification in stdexec::as_awatiable, so that
219         // should result in an awaiter that returns `std::optional`s.
220         // In the beman::execution26 impl this would mean one additor
221
222         // Option 2.a1
223         //    any sender that has a completion signature of `stdexec:
```

```
224         //    is automatically considered an awatiter that returns `s
225         // Option 2.a2
226         //    same as 2.a1, plus a mechanism to opt out (via environm
227         // Option 2.b
228         //    senders have to explicitly opt in. (via environments or
229
230         };
231
232         // Opt-In (Option 2.b)
233         OPTION2b(
234         struct env { auto query(P3570_detail::get_await_completion_ac
235         auto get_env() const noexcept -> env { return {}; }
236         ) // /OPTION2b
237
238         // common impl
239         template<stdexec::receiver rcvr> auto connect(rcvr r) { retur
240
241         coqueue<T>& queue_;
242     };
243
244     auto async_pop() -> pop_sender { return pop_sender{*this}; }
245
246     // Fake queue - returns count ints in decending order queue turns
247     auto empty() -> bool { return count_ == 0; }
248
249     auto internal_pop_() -> T { return count_--; }
250     int count_;
251 };
252
253 //-------------------------------------------------------------------
254 // Task type, ducttape version (P3552 lazy)
255 struct task{
256     struct promise_type {
257         auto initial_suspend()                  -> std::suspend_never
258         auto final_suspend()        noexcept    -> std::suspend_alway
259         [[noreturn]] auto unhandled_exception() -> void
260         auto unhandled_stopped()                -> std::coroutine_har
261
262         auto get_return_object()                -> task
263         auto return_void()                      -> void
264
265         template <stdexec::sender Awaitable>
266         auto await_transform(Awaitable&& awaitable) noexcept -> declt
```

```
267             return P3570_detail::as_awaitable(std::forward<Awaitable>
268         }
269
270         private:
271             auto tag() -> char const* { return ""; }
272     };
273 };
274
275
276 //-----------------------------------------------------------------------
277 // Example -> usage code in the coroutine
278
279 auto use(int i) -> void {   std::println("used #{}", i); }
280
281 auto coro(coqueue<int>& data) -> task {
282
283     while(auto item = co_await data.async_pop()) {
284         use(*item);
285     }
286
287     co_return;
288 }
289
290 auto main() -> int {
291     coqueue<int> queue{3};
292
293     auto _ = coro(queue);
294     std::println(" ---[done]------ ");
295
296     return EXIT_SUCCESS;
297 }
```

§ References

§ Normative References

[EXEC.AS.AWAITABLE]

[exec.as.awaitable]. 2025-02-06. URL: https://eel.is/c++draft/exec.as.awaitable

[P0260R13]

Lawrence Crowl; et al. *C++ Concurrent Queues*. 2024-12-10. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0260r13.html

§ Informative References

[EXEC26]

Dietmar Kühl. *beman.execution: Building Block For Asynchronous Programs*. 2025-01-25. URL: https://github.com/bemanproject/execution

[P3552R0]

Dietmar Kühl; Maikel Nadolski. *Add a Coroutine Lazy Type*. 2025-01-13. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3552r0.pdf